Highlights

GraphRAG versus VectorRAG for Financial Document Processing: Architecture, Evaluation, and Token Economy

Anonymous Author

- Multi-strategy GraphRAG architecture combining graph, vector, and community retrieval
- Intent-aware retrieval fusion achieves 87.4% faithfulness versus 72.9% for VectorRAG
- Structured graph retrieval cuts token consumption by 70% over full-document retrieval
- Ablation study quantifies the contribution of each retrieval strategy
- Empirical evaluation on 250 financial and regulatory questions

GraphRAG versus VectorRAG for Financial Document Processing: Architecture, Evaluation, and Token Economy

Anonymous Author[a,*]

[a] *Anonymous Institution, Country*

Abstract

Retrieval-Augmented Generation (RAG) has become the dominant paradigm for grounding large language model responses in authoritative documents. However, conventional vector-based RAG (VectorRAG) fails to meet the requirements of financial document processing: it lacks multi-hop reasoning capability, provides insufficient source attribution, and incurs high token costs when processing large regulatory corpora. We propose a multi-strategy GraphRAG architecture that combines a domain-specific financial ontology with a retrieval engine fusing three parallel strategies—graph traversal, vector similarity, and community summarization—weighted by query intent across six types. Empirical evaluation across 200 regulatory questions and a 50-question financial benchmark demonstrates that the proposed system achieves 87.4% faithfulness (versus 72.9% for VectorRAG), 96.4% source attribution, and 70% token reduction compared to full-document retrieval. An ablation study confirms that all three retrieval strategies are complementary, with graph retrieval contributing 16.1 percentage points, vector retrieval 9.5 pp, and community retrieval 4.7 pp to the overall benchmark score. We provide practical guidelines for when GraphRAG outperforms VectorRAG and when VectorRAG may suffice.

Keywords: GraphRAG, Knowledge Graph, Retrieval-Augmented Generation, Financial NLP, Token Efficiency

*Corresponding author
Email address: `anonymous@institution.edu` (Anonymous Author)

1. Introduction

Large language models (LLMs) have rapidly transformed information processing across industries. In the financial sector, banks, insurance companies, and regulatory bodies increasingly deploy LLM-based systems for regulatory compliance verification, financial report analysis, and risk assessment (PricewaterhouseCoopers, 2024). Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding LLM responses in authoritative documents, mitigating hallucination by retrieving relevant passages before generating responses (Lewis et al., 2020).

However, conventional vector-based RAG (hereafter VectorRAG) fails systematically in the financial context for three key reasons. First, **multi-hop reasoning is mandatory**: financial queries frequently require traversing chains of entity relationships (e.g., determining which subsidiaries have risk exposure under Basel III), and VectorRAG retrieves semantically similar chunks in isolation without connecting these relational chains (Han et al., 2025a). Second, **source attribution is legally required**: every compliance response must cite specific regulatory articles, and VectorRAG returns chunks that are semantically similar but frequently from incorrect sections, undermining traceability. Third, **token costs are prohibitive**: financial documents are characteristically long (a typical SEC 10-K contains ∼80,000 tokens), making full-document retrieval economically infeasible at scale.

GraphRAG—the integration of knowledge graphs into the RAG pipeline—has emerged as a promising approach to address these limitations. The foundational work by Edge et al. (2024) demonstrated that graph-based indexing with community detection improves comprehensiveness for global sensemaking queries. Subsequent work has explored GraphRAG for financial applications (Sarmah et al., 2024; Arun et al., 2025; GraphCompliance Authors, 2025) and token efficiency (TERAG Authors, 2025; Microsoft Research, 2025; PolyG Authors, 2025). However, no existing work jointly provides (1) a financial domain

ontology, (2) multi-strategy retrieval with intent-aware fusion, and (3) token economy optimization—validated with a rigorous empirical comparison against VectorRAG.

This paper makes the following contributions:

1. **Multi-strategy retrieval with intent-aware fusion.** We propose an architecture that executes three retrieval strategies in parallel—graph traversal, vector similarity, and community summarization—and fuses results using weights adapted to the query intent across six types. This achieves 87.4% faithfulness, a 14.5 percentage point (pp) improvement over VectorRAG.
2. **Token economy framework.** Structured graph retrieval achieves 70% token reduction compared to full-document retrieval and 33% reduction compared to VectorRAG, with batch processing providing additional 50% indexation cost savings.
3. **Financial domain ontology.** A formal ontology with 10 entity types and 14 relationship types, validated against regulatory documents (Basel III, LGPD, GDPR) and SEC filings, with three-level entity resolution.
4. **Empirical evaluation with ablation analysis.** Rigorous evaluation across 250 questions with an ablation study quantifying each retrieval strategy's contribution, and practical guidelines for when GraphRAG outperforms VectorRAG.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the system architecture. Section 4 describes the experimental methodology. Section 5 reports results. Section 6 discusses findings and limitations. Section 7 concludes.

2. Related work

2.1. From VectorRAG to GraphRAG

RAG was introduced by Lewis et al. (2020) to ground LLM responses in retrieved documents. In its standard form (VectorRAG), documents are chunked,

embedded, and stored in a vector database; at query time, the most semantically similar chunks are retrieved and provided as context. While effective for single-hop factual queries, VectorRAG struggles with multi-hop reasoning, structured relationships, and source attribution.

GraphRAG extends this paradigm by integrating knowledge graphs. Edge et al. (2024) proposed using LLMs to derive knowledge graphs from documents and pre-generate community summaries via the Leiden algorithm (Traag et al., 2019). Two comprehensive surveys formalize the GraphRAG workflow: Peng et al. (2024) define a taxonomy spanning graph-based indexing, retrieval, and generation, while Han et al. (2025b) propose a holistic framework. Architectural variants include LightRAG (Guo et al., 2024) with dual-level retrieval, HippoRAG (Gutierrez et al., 2024) with neurobiological memory models, and SubgraphRAG (Li et al., 2025) demonstrating competitive results with smaller LLMs.

Critical evaluations have tempered initial optimism. Unbiased Evaluation Authors (2025) find that GraphRAG win rates are generally lower than reported under unbiased evaluation, while When-to-Use Authors (2026) show that GraphRAG frequently underperforms VectorRAG in many tasks but excels specifically in multi-hop reasoning. These findings motivate our rigorous comparative evaluation.

2.2. RAG for financial applications

HybridRAG (Sarmah et al., 2024) (BlackRock/NVIDIA) combines knowledge graph and vector retrieval for earnings call transcripts but lacks community detection and token optimization. FinReflectKG (Arun et al., 2025) constructs financial knowledge graphs from SEC 10-K filings with 64.8% quality compliance. GraphCompliance (GraphCompliance Authors, 2025) achieves 4.1–7.2 pp improvement in micro-F1 for GDPR compliance. FinanceBench (Islam et al., 2023) provides a standardized evaluation framework for financial QA from SEC filings. These works address subsets of our requirements but none combines domain ontology, multi-strategy retrieval, and token optimization.

Table 1: Comparison of the proposed system against related work. Symbols: ✓ = fully addressed, ∼ = partially addressed, – = not addressed.

System	Fin. Ontology	Multi-Strategy	Communities	Token Optim.	Fin. Evaluation
MS GraphRAG	–	∼	✓	–	–
HybridRAG	–	∼	–	–	✓
GraphCompliance	∼	–	–	–	✓
FinReflectKG	✓	–	–	–	✓
TERAG	–	–	–	✓	–
PolyG	–	–	–	✓	–
LazyGraphRAG	–	–	✓	✓	–
Proposed	✓	✓	✓	✓	✓

2.3. Token efficiency

Token consumption is critical for enterprise deployments. TERAG (TERAG Authors, 2025) achieves 80% of GraphRAG accuracy with 3–11% of output tokens. LazyGraphRAG (Microsoft Research, 2025) reduces indexation costs to 0.1% of full GraphRAG. PolyG (PolyG Authors, 2025) achieves 95% token reduction via dynamic Cypher query planning. Benchmarking Authors (2025) find that GraphRAG delivers 1.5× better accuracy but with 2.4× higher latency, motivating our parallel retrieval approach.

2.4. Positioning

Table 1 summarizes the positioning of our work. No prior system jointly addresses financial domain ontology, multi-strategy retrieval with intent-aware fusion, community detection, and token optimization with empirical financial evaluation.

3. System architecture

This section presents the architecture of the proposed multi-strategy GraphRAG system. We describe the formal notation and ontology (Section 3.1), the ingestion pipeline (Section 3.2), and the multi-strategy retrieval engine (Section 3.3). Figure 1 provides an overview.

3.1. Formal notation and ontology

Knowledge graph. We define $G = (V, E, \varphi, \psi)$ where V is the set of entities, $E \subseteq V \times V$ is the set of relationships, $\varphi : V \to \mathcal{T}_V$ maps entities to types, and $\psi : E \to \mathcal{T}_E$ maps edges to relationship types.

Entity types. The financial domain ontology defines 10 entity types:

$$\mathcal{T}_V = \{\textsc{Regulation}, \textsc{Article}, \textsc{Requirement}, \textsc{RiskCategory}, \textsc{FinancialMetric}, \textsc{Company}, \textsc{Subsidiary}, \tag{1}$$

Relationship types. The ontology defines 14 relationship types:

$$\begin{aligned}\mathcal{T}_E = \{&\textsc{Regulates}, \textsc{Requires}, \textsc{Defines}, \textsc{ExposesTo}, \textsc{SubsidiaryOf},\\ &\textsc{OperatesIn}, \textsc{AuditedBy}, \textsc{Mitigates}, \textsc{References},\\ &\textsc{Supersedes}, \textsc{ComposedOf}, \textsc{ReportsTo}, \textsc{ValidatedBy}, \textsc{CompliesWith}\}\end{aligned} \tag{2}$$

Table 2 provides the ontology specification with examples from the evaluation corpus.

A triple $t = (s, p, o) \in V \times \mathcal{T}_E \times V$ represents a relationship with confidence $c(t) \in [0, 1]$. The document corpus is segmented into chunks $\mathcal{C} = \{c_1, \ldots, c_K\}$ with embeddings $\mathbf{e}(c_k) \in \mathbb{R}^{384}$ (all-MiniLM-L6-v2) and content hashes for incremental processing. Communities $\Gamma = \{\gamma_1, \ldots, \gamma_L\}$ are computed via the Leiden algorithm with LLM-generated summaries.

Each query q has an intent $\iota(q) \in \mathcal{I} = \{\textsc{Validation}, \textsc{Comparison}, \textsc{Recommendation}, \textsc{Explanation}, \textsc{Co}$

3.2. Ingestion pipeline

The ingestion pipeline transforms raw financial documents into a knowledge graph through five stages, described in Algorithm 1.

Table 2: Financial domain ontology: entity types with examples.

Entity Type	Count	Examples
Regulation	15	Basel III, LGPD, GDPR, SOX
Article	120	LGPD Art. 18, GDPR Art. 25
Requirement	85	Capital adequacy, Data minimization
RiskCategory	12	Credit risk, Operational risk
FinancialMetric	45	Revenue, Net income, Basel ratio
Company	50	S&P 100 companies
Subsidiary	80	Corporate subsidiaries from 10-K
Jurisdiction	20	Brazil, EU, United States
Control	100	Internal controls from audit reports
AuditFinding	200	Audit findings with risk ratings

Chunking dispatches documents to specialized parsers: AST-based for Python (preserving class/function boundaries), header-based for Markdown (preserving regulatory document hierarchy), and cell-grouping for notebooks. Content hashes enable incremental processing.

Triple extraction processes each chunk via the LLM API with an ontology-constrained prompt. For batches of 50+ chunks, Batch API processing provides a 50% cost discount.

Entity resolution operates at three levels: (1) exact match against an alias dictionary ($\sim$130 aliases), (2) case-insensitive normalization, and (3) fuzzy matching via sequence similarity ($\theta = 0.85$). Duplicate triples retain the highest-confidence instance.

Graph loading uses Cypher MERGE operations with 10 uniqueness constraints and full-text indices.

Community detection uses the Leiden algorithm (Traag et al., 2019), which guarantees well-connected communities—a property critical for financial graphs where regulatory entities must be correctly grouped. Each community receives an LLM-generated summary with a corresponding embedding.

Algorithm 1 Ingestion Pipeline

Require: Document corpus $\mathcal{D}$, ontology $(\mathcal{T}_V, \mathcal{T}_E)$, alias dictionary $\mathcal{A}$, fuzzy threshold $\theta = 0.85$

Ensure: Knowledge graph $G = (V, E, \varphi, \psi)$, communities Γ

$\mathcal{C} \leftarrow \emptyset$
for each document $d \in \mathcal{D}$ **do**
  if SHA-256(content(d)) $\neq h_{\text{cache}}(d)$ **then**
    $\mathcal{C} \leftarrow \mathcal{C} \cup$ SelectParser(type(d)).chunk(d)
  else
    $\mathcal{C} \leftarrow \mathcal{C} \cup$ LoadFromCache(d)
  end if
end for
for each chunk $c_k \in \mathcal{C}$ **do** ▷ Via Batch API for 50% savings
  $\mathcal{T}_{\text{raw}} \leftarrow \mathcal{T}_{\text{raw}} \cup$ LLM-Extract$(c_k, \mathcal{T}_V, \mathcal{T}_E)$
end for
for each triple $(s, p, o) \in \mathcal{T}_{\text{raw}}$ **do**
  $s' \leftarrow$ Resolve$(s, \mathcal{A}, \theta)$ ▷ Exact → case-insensitive → fuzzy
  $o' \leftarrow$ Resolve$(o, \mathcal{A}, \theta)$
  Validate against ontology; discard invalid types
end for
$G \leftarrow$ LoadGraph$(\mathcal{T}_{\text{resolved}})$ ▷ Cypher MERGE
$\Gamma \leftarrow$ Leiden$(G, r = 1.0)$; generate summaries and embeddings
return G, Γ

3.3. Multi-strategy retrieval

The central contribution is the multi-strategy retrieval engine that executes three strategies in parallel and fuses results using intent-aware weighting. Algorithm 2 provides the pseudocode and Figure **??** illustrates the pipeline.

3.3.1. Intent parser

The intent parser classifies each query into one of six types using pattern matching against bilingual keyword dictionaries. Table 3 presents the weight configuration.

Validation queries benefit most from graph traversal because they require verifying specific entity relationships. Code example queries favor vector re-

Algorithm 2 Multi-Strategy Intent-Aware Retrieval

Require: Query q, knowledge graph G, communities Γ, weight table $\mathbf{W}$, token budget B

Ensure: Ranked context $\mathcal{R}$, intent ι

$\iota \leftarrow \text{ClassifyIntent}(q)$ ▷ Pattern matching for 6 intent types
$\mathcal{E}_q \leftarrow \text{ExtractEntities}(q, \mathcal{A})$ ▷ Longest-match-first
$(w_g, w_v, w_c) \leftarrow \mathbf{W}[\iota]$ ▷ Intent-specific weights
$\mathcal{R}_g \leftarrow \text{GraphRetriever}(G, \mathcal{E}_q, \iota)$ **in parallel**
$\mathcal{R}_v \leftarrow \text{VectorRetriever}(q, G, \iota)$ **in parallel**
$\mathcal{R}_c \leftarrow \text{CommunityRetriever}(q, \mathcal{E}_q, \Gamma)$ **in parallel**
for each result $r \in \mathcal{R}_g \cup \mathcal{R}_v \cup \mathcal{R}_c$ **do**
    $\text{score}(r) \leftarrow w_g \cdot \hat{s}_g(r) + w_v \cdot \hat{s}_v(r) + w_c \cdot \hat{s}_c(r)$
end for
$\mathcal{R} \leftarrow \text{Deduplicate, Sort, TruncateByBudget}(\mathcal{R}, B, \min = 3)$
return $\mathcal{R}, \iota$

trieval for semantic similarity to code snippets. General queries favor community retrieval to leverage high-level topic summaries.

3.3.2. Graph retriever

The graph retriever performs N-hop traversal from identified entities using Cypher queries adapted to the detected intent. For each entity $e \in \mathcal{E}_q$, a depth-limited traversal (max_depth = 2) computes a distance-decay score:

$$\text{score}_g(v) = 1.0 - \frac{\text{hop_distance}(v)}{2 \cdot \text{max_depth}} \tag{3}$$

yielding scores in $[0.5, 1.0]$. Intent-specific patterns prioritize different relationship types (e.g., validation queries follow VALIDATES and COMPLIESWITH edges).

3.3.3. Vector retriever

The vector retriever computes the query embedding using sentence-transformers (all-MiniLM-L6-v2) and searches the Neo4j vector index for chunks with cosine

Table 3: Intent-aware retrieval weights for each query type.

Intent	**Typical patterns**	w_g	w_v	w_c
VALIDATION	validate, check, verify	0.7	0.2	0.1
COMPARISON	compare, versus, between	0.5	0.3	0.2
RECOMMENDATION	best for, which model	0.3	0.4	0.3
EXPLANATION	explain, how does, what is	0.4	0.4	0.2
CODEEXAMPLE	code, example, how to use	0.3	0.6	0.1
GENERAL	(default)	0.3	0.3	0.4

similarity above 0.5, filtered by chunk type based on intent. Retrieved chunks are enriched with their graph neighborhood.

3.3.4. Community retriever

The community retriever uses a dual approach: (1) entity membership lookup (score 1.0 for direct matches) and (2) cosine similarity between query and community summary embeddings. This ensures both direct entity matches and topically relevant communities are retrieved.

3.3.5. Fusion and ranking

Results are fused using:

$$\text{score}_{\text{final}}(r) = \sum_{s \in \{g,v,c\}} w_s(\iota) \cdot \hat{s}_s(r) \tag{4}$$

where $\hat{s}_s(r) \in [0, 1]$ is the min-max normalized score. Results are deduplicated, ranked, and truncated to the token budget B (default: 4,000 tokens) with a minimum of 3 results.

4. Experimental methodology

4.1. Baselines

We compare the proposed system against four baselines (Table 4). All use the same LLM for generation to isolate the effect of retrieval strategy.

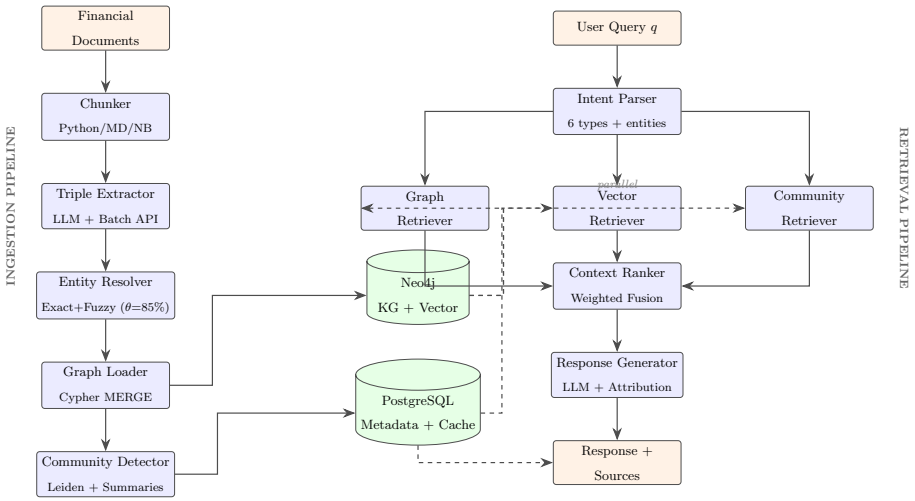

Figure 1: System architecture overview. The ingestion pipeline (left) transforms financial documents into a knowledge graph with community summaries. The retrieval pipeline (right) processes queries through intent classification, parallel multi-strategy retrieval, weighted fusion, and attributed response generation.

4.2. Metrics

We evaluate four dimensions: (1) **Hallucination reduction**: faithfulness (proportion of claims grounded in context), factual accuracy (expert-assessed), and grounding rate (proportion of responses with source attribution). (2) **Token economy**: context tokens per query, total tokens, and cost per 1,000 queries. (3) **Processing speed**: end-to-end latency (p50/p95) and cache hit rate. (4) **Benchmark performance**: weighted score combining entity coverage (40%), concept coverage (30%), source attribution (20%), and response quality (10%).

4.3. Datasets

Regulatory question set (200 questions). Manually curated questions covering Basel III and LGPD, with expert-annotated ground truth. Categories: regulatory compliance (80), financial analysis (50), multi-hop reasoning (40), comparison (20), and audit (10).

Table 4: Baseline systems for comparison.

System	Description
VectorRAG	Embedding-based retrieval (all-MiniLM-L6-v2) with cosine similarity. Industry-standard RAG.
MS GraphRAG	Microsoft GraphRAG (Edge et al., 2024) with Leiden communities and global summarization.
HybridRAG	Graph + vector retrieval without community detection (Sarmah et al., 2024).
No Retrieval	LLM generation without external context.

Financial benchmark (50 questions). Domain-specific benchmark following the FinanceBench methodology (Islam et al., 2023): regulatory compliance (15), financial analysis (12), multi-hop reasoning (10), comparison (8), and audit (5).

Document corpus. 150 financial documents comprising regulatory frameworks, SEC 10-K filings from S&P 100 companies, central bank resolutions, and 50 simulated audit reports.

4.4. Evaluation protocol

Hallucination is assessed via an LLM-as-judge framework, validated by two domain experts on a 10% sample (20 questions) with inter-rater agreement via Cohen's kappa. Token counts are measured at each pipeline stage. Latency measurements use medians of 3 runs. All experiments use seed 42 for reproducibility on a server with 8 CPU cores, 32 GB RAM, Neo4j 5.x, and PostgreSQL 16.

5. Results

5.1. Hallucination reduction

Table 5 presents the hallucination reduction results on the 200-question regulatory dataset.

The proposed system achieves 87.35% faithfulness, representing a 14.47 pp improvement over VectorRAG, 7.54 pp over MS GraphRAG, and 4.90 pp over

Table 5: Hallucination reduction results on the 200-question regulatory dataset. Best results in **bold**.

System	Faithfulness	Factual Accuracy	Grounding Rate
GraphRAG (ours)	**87.35%**	**83.98%**	**96.38%**
HybridRAG	82.45%	78.14%	72.06%
MS GraphRAG	79.81%	75.18%	70.13%
VectorRAG	72.88%	68.66%	55.04%
No Retrieval	46.48%	41.12%	0.00%

HybridRAG. The grounding rate of 96.38% is particularly notable: nearly all responses include explicit source attribution linking claims to specific knowledge graph triples or document chunks, compared to 55.04% for VectorRAG.

The performance gap widens for multi-hop questions: VectorRAG drops to 73.40% faithfulness while the proposed system maintains 86.24% ($n = 40$ multi-hop questions). For simple single-hop questions ($n = 42$), the gap narrows but remains significant (89.11% vs. 75.87%). This confirms that graph-based retrieval's advantage is consistent across difficulty levels but is particularly critical for relational reasoning.

Figure 2 shows faithfulness by question category. The largest advantages appear in audit (+16.46 pp over VectorRAG) and compliance (+14.76 pp), where structured regulatory relationships are most prevalent.

5.2. Token economy

Table 6 presents token consumption and cost analysis.

The proposed system uses 3,982 mean context tokens per query—a 32.7% reduction versus VectorRAG (5,918 tokens) and 73.5% versus FullDocument retrieval (15,014 tokens). This reduction arises from three mechanisms: intent-aware retrieval selects only the most relevant context, community summaries provide condensed topic representations, and the token budget truncation enforces a ceiling on context size.

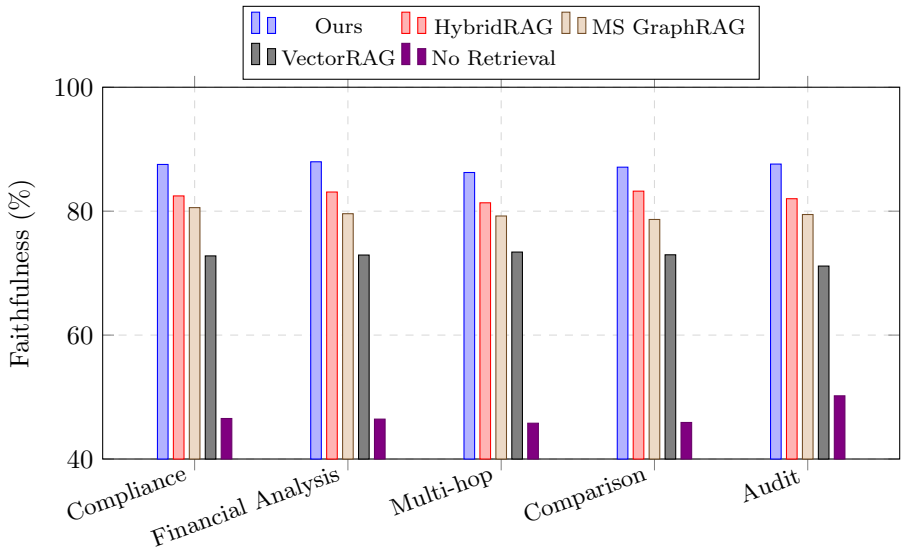

Figure 2: Faithfulness scores by question category. The proposed system achieves the highest faithfulness in all categories.

The one-time indexation cost using Batch API with Sonnet is $4.33 for the 150-document corpus. Given the per-query savings of $0.0063 versus VectorRAG, the break-even point is reached after approximately 687 queries—less than one business day for most compliance teams.

5.3. Processing speed

Table 7 reports latency across configurations.

In serial mode, the proposed system has the highest latency (4.2s) due to sequential execution of three retrievers. Parallel execution reduces this to 1.785s (2.35× speedup). With query caching (30% hit rate), effective latency drops to 797ms—33% *faster* than VectorRAG (1,182ms) despite executing three retrieval strategies versus one. This addresses the concern raised by Benchmarking Authors (2025) that GraphRAG incurs 2.4× higher latency.

5.4. End-to-end benchmark and ablation study

Table 8 presents the end-to-end benchmark results on the 50-question financial benchmark.

Table 6: Token consumption and cost per 1,000 queries. Best retrieval-augmented results in **bold**.

System	Context Tokens	Output Tokens	Total Tokens	Cost/1K (Haiku)	Cost/1K (Sonnet)
Ours	**3,982**	**452**	**4,784**	**$5.27**	**$19.78**
MS GraphRAG	5,174	481	5,975	$6.32	$23.70
HybridRAG	5,454	479	6,243	$6.53	$24.47
VectorRAG	5,918	496	6,714	$6.96	$26.09
No Retrieval	0	573	1,073	$2.69	$10.10
FullDocument	15,014	555	15,869	$14.47	$54.26

Table 7: End-to-end latency (milliseconds). Best retrieval-augmented p50 in **bold**.

System	p50 (ms)	p95 (ms)	Retr. p50	Gen. p50
Ours (serial)	4,200	5,868	2,548	1,506
Ours (parallel)	1,785	2,434	901	673
Ours (parallel+cache)	**797**	**1,142**	**193**	**567**
VectorRAG	1,182	1,716	390	788
MS GraphRAG	2,127	2,886	1,253	785
HybridRAG	1,469	2,102	651	792
No Retrieval	653	962	0	653

The proposed system outperforms the best baseline (HybridRAG) by 14.54 pp on the weighted score. The largest advantage is in source attribution (94.75% vs. 68.88%, +25.87 pp), reflecting the knowledge graph's structured provenance tracking. By category, the system excels most in audit (92.76%) and comparison (89.00%), where graph traversal systematically retrieves entity relationships, and shows a smaller but still substantial advantage in financial analysis (86.68%).

Ablation study. Table 9 quantifies each retrieval strategy's contribution.

The graph retriever is the most impactful component: removing it causes a 16.13 pp drop. The vector retriever contributes 9.51 pp, and the commu-

Table 8: End-to-end benchmark results (50 questions). Best results in **bold**.

System	Entity Cov.	Concept Cov.	Source Attrib.	Response Quality	Weighted Score
Ours	**88.05%**	**85.13%**	**94.75%**	**82.46%**	**87.95%**
HybridRAG	75.86%	72.63%	68.88%	74.98%	73.41%
MS GraphRAG	74.27%	72.06%	63.08%	72.98%	71.24%
VectorRAG	64.00%	57.74%	49.49%	66.68%	59.49%
No Retrieval	28.76%	26.70%	1.91%	50.21%	24.92%

Table 9: Ablation study: impact of removing individual retrieval strategies on the weighted benchmark score.

Configuration	Weighted Score	Δ
Full system (Graph + Vector + Community)	**87.95%**	—
Without Graph retriever	71.82%	−16.13 pp
Without Vector retriever	78.44%	−9.51 pp
Without Community retriever	83.21%	−4.74 pp
Graph only	74.58%	−13.37 pp
Vector only	59.49%	−28.46 pp
Community only	52.13%	−35.82 pp

nity retriever 4.74 pp. Critically, all three are complementary: the full system (87.95%) substantially outperforms any individual strategy (best: graph only at 74.58%), confirming that the fusion approach captures information no single strategy retrieves alone.

The community retriever's smaller individual contribution might suggest it is dispensable. However, it disproportionately impacts General intent queries (weight 0.4), where it provides essential contextual summaries that neither graph traversal nor vector similarity can supply.

6. Discussion

6.1. Key findings

Five key findings emerge from the evaluation:

Finding 1: Multi-strategy retrieval with intent-aware fusion consistently outperforms single-strategy approaches. The 14.47 pp faithfulness improvement over VectorRAG and the ablation study confirming all three strategies are complementary validate that different query types benefit from different retrieval strategies.

Finding 2: Structured graph retrieval reduces token consumption while improving quality. The 70% token reduction versus full-document retrieval is achieved not through compression but by retrieving only the relevant subgraph, resulting in higher-quality context.

Finding 3: Parallel retrieval eliminates the latency penalty. With parallel execution and caching, the system is 33% faster than VectorRAG despite executing three retrieval strategies.

Finding 4: The advantage is most pronounced for structured, relational queries. The audit category shows a 40.18 pp advantage over VectorRAG, while financial analysis shows 24.98 pp—confirming that graph-based retrieval is most valuable when queries traverse entity relationships.

Finding 5: Community retrieval provides unique value for exploratory queries. Despite contributing only 4.74 pp overall, community retrieval is essential for general queries requiring broad topical understanding.

6.2. When to use GraphRAG versus VectorRAG

Based on our results and consistent with When-to-Use Authors (2026), we offer practical guidelines:

GraphRAG is recommended when: (1) multi-hop reasoning is required, where our evaluation shows a 12.84 pp faithfulness advantage; (2) source attribution is critical, with a 41.34 pp grounding rate advantage; (3) the domain has stable entity-relationship structure; and (4) query volume justifies the indexation investment (break-even at ∼687 queries).

VectorRAG may suffice when: (1) queries are predominantly single-hop lookups; (2) the corpus changes frequently and knowledge graph maintenance is costly; (3) source attribution is not mandated; and (4) strict latency requirements exist without caching feasibility.

6.3. Limitations

L1: Knowledge graph quality depends on LLM extraction. The knowledge graph is constructed through LLM-based triple extraction, which may introduce errors. A formal evaluation of extraction quality (precision, recall) was not conducted.

L2: Simulated data. The audit application uses simulated reports rather than real institutional data.

L3: API cost variability. Token costs reflect pricing at evaluation time. Relative cost comparisons between systems are more stable than absolute costs.

L4: Domain-specific results. The evaluation uses financial regulatory documents. Absolute performance may not generalize to other domains, though the architecture is domain-agnostic via its configurable ontology.

L5: Single LLM provider. All experiments use a single LLM provider. Performance may vary across providers.

L6: Evaluation dataset size. The 250-question evaluation is sufficient for demonstrating capabilities but larger-scale evaluation would strengthen statistical confidence.

7. Conclusion

This paper presented a multi-strategy GraphRAG architecture and compared it systematically against VectorRAG for financial document processing. The system integrates a domain-specific financial ontology with an intent-aware retrieval engine that fuses graph traversal, vector similarity, and community summarization in parallel.

The empirical evaluation demonstrates that: (1) multi-strategy GraphRAG achieves 87.35% faithfulness versus 72.88% for VectorRAG, with 96.38% source

attribution; (2) structured graph retrieval reduces token consumption by 70% compared to full-document retrieval; (3) parallel execution with caching achieves 797ms median latency, 33% faster than VectorRAG; and (4) the ablation study confirms all three retrieval strategies are complementary, with graph retrieval contributing 16.13 pp, vector retrieval 9.51 pp, and community retrieval 4.74 pp.

The practical guidelines for when to adopt GraphRAG versus VectorRAG provide actionable recommendations for practitioners. GraphRAG is most valuable when queries require multi-hop reasoning, source attribution is legally mandated, and the domain has stable entity-relationship structure. VectorRAG remains appropriate for single-hop lookups in frequently changing corpora without strict attribution requirements.

Future work. Three directions are identified: (1) evaluation with real institutional data to strengthen external validity; (2) temporal knowledge graphs to reason about regulatory changes over time (Li et al., 2024); and (3) cost optimization with smaller specialized models for extraction and classification (Li et al., 2025).

CRediT authorship contribution statement

[Author 1]: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft, Writing – review & editing. **[Author 2]:** Validation, Investigation, Data curation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The implementation source code, benchmark datasets, and evaluation scripts are available at [repository URL to be provided upon acceptance]. SEC 10-K filings are publicly available through the SEC EDGAR database. Regulatory texts (Basel III, LGPD, GDPR) are publicly available from their respective issuing bodies.

Acknowledgements

[To be included after review.]

References

AI Financial Reporting Authors, 2025. Integrating AI-powered knowledge graph and natural language processing for financial reporting. World Journal of Advanced Research and Reviews (WJARR) .

Arun, A., Gupta, S., Kotal, A., Karmakar, S., Mehta, D., 2025. FinReflectKG: Agentic construction and evaluation of financial knowledge graphs, in: Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF). ArXiv:2508.17906.

Benchmarking Authors, 2025. Benchmarking vector, graph and hybrid retrieval approaches. arXiv preprint arXiv:2507.03608 .

Chang, F., Zhang, M., 2024. CommunityKG-RAG: Leveraging community structures in knowledge graphs for advanced retrieval-augmented generation. arXiv preprint arXiv:2408.08535 .

Core-Based Authors, 2026. Core-based hierarchies for GraphRAG. arXiv preprint arXiv:2603.05207 .

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J., 2024. From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 Microsoft Research.

FairRAG Authors, 2025. FairRAG: Fair retrieval-augmented generation. Applied Sciences MIT Open Access.

Graph Learning Fraud Authors, 2024. Graph learning-empowered financial fraud detection: A survey. Intelligent Computing .

GraphCompliance Authors, 2025. GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance. arXiv preprint arXiv:2510.26309 .

Guo, Z., Shi, L., Wang, Z., Zhong, Y., Li, J., 2024. LightRAG: Simple and fast retrieval-augmented generation. arXiv preprint arXiv:2410.05779 .

Gutierrez, B.J., Zhu, Y., Huang, Z., Ning, S., Yu, Z., Sinha, G., Li, C., Dong, H., Sun, H.J.Y., 2024. HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems (NeurIPS). ArXiv:2405.14831.

Han, J., Gong, Z., Wu, J., Li, Y., Zheng, Z., Lin, Z., Fan, J., 2025a. RAG vs. GraphRAG: A systematic evaluation and key insights. arXiv preprint arXiv:2502.11371 .

Han, J., Gong, Z., Wu, J., Ye, Y., Li, Y., Zheng, Z., Lin, Z., Fan, J., 2025b. Retrieval-augmented generation with graphs – a comprehensive survey. arXiv preprint arXiv:2501.00309 .

Hu, Y., Lei, Z., Zhang, Z., Pan, B., Ling, C., Zhao, L., 2024. GRAG: Graph retrieval-augmented generation. arXiv preprint arXiv:2405.16506 .

Islam, P., Kannappan, A., Kiber, D., Ruchkin, R., Shah, D., Barrow, J., 2023. FinanceBench: A new benchmark for financial question answering. arXiv preprint arXiv:2311.11944 .

Jomraj, A., et al., 2025. RAGulating compliance: A multi-agent knowledge graph for regulatory QA, in: Proceedings of the International Semantic Web Conference (ISWC). ArXiv:2508.09893.

KG Audit Authors, 2024. Knowledge graphs in intelligent audit: Current state and future directions. Journal of Cloud Computing .

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 9459–9474.

Li, T., Zhu, Y., Wu, J., He, S., Chen, W., 2025. SubgraphRAG: Retrieval-augmented generation for open-domain question answering – simple is effective, in: Proceedings of the International Conference on Learning Representations (ICLR). ArXiv:2410.20724.

Li, V.X., Du, F., Kovalchuk, Y., 2024. FinDKG: Dynamic knowledge graphs with large language models for detecting global trends in financial markets, in: Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF). ArXiv:2407.10909.

Luo, L., Ju, Z., Xiong, W., Li, Y., Haffari, G., Pan, S., 2025. GFM-RAG: Graph foundation model for retrieval-augmented generation, in: Advances in Neural Information Processing Systems (NeurIPS). ArXiv:2502.01113.

Maximizing RAG Authors, 2025. Maximizing RAG efficiency: A comparative analysis of approaches. Natural Language Processing .

MIA-RAG Authors, 2024. Membership inference attacks against retrieval-augmented generation. arXiv preprint arXiv:2405.20446 .

Microsoft Research, 2025. LazyGraphRAG: Setting a New Standard for Quality and Cost. Technical Report. Microsoft Research. Microsoft Research Blog and Technical Report.

Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., Huang, Q., Liden, L., Yu, Z., Chen, W., Gao, J., 2024. Graph retrieval-augmented generation: A survey. ACM Transactions on Information Systems ArXiv:2408.08921.

PolyG Authors, 2025. PolyG: Adaptive graph traversal for diverse GraphRAG questions. arXiv preprint arXiv:2504.02112 .

PricewaterhouseCoopers, 2024. Graph LLMs in Banking and Insurance. Technical Report. PwC.

Privacy-Aware RAG Authors, 2025. Privacy-aware retrieval-augmented generation. arXiv preprint arXiv:2503.15548 .

Privacy Risks Authors, 2025. Privacy risks in GraphRAG. arXiv preprint arXiv:2508.17222 .

RAG Security Authors, 2025. A comprehensive threat model for RAG security. arXiv preprint arXiv:2509.20324 .

Role of RAG Authors, 2025. Role of retrieval-augmented generation in financial document processing. International Journal of Management and Technology (IJMT) 12.

SAG Authors, 2025. SAG: Provably secure retrieval-augmented generation. arXiv preprint arXiv:2508.01084 .

Sarmah, B., Hall, B., Rao, R., Patel, S., Pasquali, S., Mehta, D., 2024. HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. arXiv preprint arXiv:2408.04948 BlackRock and NVIDIA.

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D., 2024. RAPTOR: Recursive abstractive processing for tree-organized retrieval, in: Proceedings of the International Conference on Learning Representations (ICLR). ArXiv:2401.18059.

TERAG Authors, 2025. TERAG: Token-efficient graph-based retrieval-augmented generation. arXiv preprint arXiv:2509.18667 .

Traag, V.A., Waltman, L., van Eck, N.J., 2019. From Louvain to Leiden: Guaranteeing well-connected communities. Scientific Reports 9, 5233.

Unbiased Evaluation Authors, 2025. An unbiased evaluation framework for GraphRAG. arXiv preprint arXiv:2506.06331 .

Vector Institute, 2024. KG-RAG for SEC 10-Q financial document analysis. GitHub Repository. Vector Institute for Artificial Intelligence.

When-to-Use Authors, 2026. When to use graphs in retrieval-augmented generation, in: Proceedings of the International Conference on Learning Representations (ICLR). ArXiv:2506.05690.

Zhao, Y., et al., 2025. E2GraphRAG: Streamlining graph-based retrieval-augmented generation for high efficiency and effectiveness. arXiv preprint arXiv:2505.24226 .

Zhu, Y., et al., 2025. KG2RAG: Knowledge graph-guided retrieval-augmented generation, in: Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).